

ABSTRACT

This project presents the design and implementation of a cloud-based weather data pipeline that collects, processes, stores, and analyzes weather information from multiple cities. The system uses the OpenWeather API to fetch real-time weather data, including temperature, humidity, weather conditions, and timestamps. AWS Lambda is utilized to automate data collection and processing, while Amazon DynamoDB serves as the primary storage database. DynamoDB Streams trigger an additional Lambda function to validate incoming records and classify them as clean or rejected data. The processed data is stored in Amazon S3, which acts as a centralized data lake. Snowflake is then used as a cloud data warehouse to load, transform, and manage the weather data for analytical purposes. Finally, Python libraries such as Pandas and Matplotlib are used to visualize the data through charts and graphs. This project demonstrates an end-to-end data engineering workflow involving cloud computing, data warehousing, and data analytics technologies.

INTRODUCTION

With the increasing demand for real-time data processing and cloud-based analytics, organizations require scalable solutions to collect, store, and analyze large volumes of data efficiently. Weather data is widely used in various sectors such as agriculture, transportation, tourism, and disaster management. The objective of this project is to develop a complete weather data pipeline that automates the collection of weather information from external APIs and processes it using modern cloud technologies.

The project integrates multiple AWS services such as Lambda, DynamoDB, DynamoDB Streams, and S3 with Snowflake, a cloud-based data warehouse. The collected weather data is validated, stored, transformed, and analyzed to generate meaningful insights. The implementation demonstrates practical applications of data engineering concepts, including ETL processes, cloud storage, data validation, and visualization techniques.

Weather information is generated continuously and requires an efficient mechanism for collection, storage, validation, and analysis. Traditional systems often struggle with scalability, automation, and real-time processing capabilities. There is a need for a cloud-based solution that can automatically collect weather data from external sources, validate the data, store it securely, and provide analytical capabilities for decision-making. This project aims to address these challenges by building a scalable and automated weather data pipeline using AWS and Snowflake technologies.

OBJECTIVES

1. To collect real-time weather data using the OpenWeather API.
2. To automate weather data ingestion using AWS Lambda.
3. To store weather records in Amazon DynamoDB.
4. To validate incoming weather records and identify invalid or duplicate data.
5. To store processed data in Amazon S3 for long-term retention.
6. To integrate Amazon S3 with Snowflake for data warehousing.
7. To transform semi-structured JSON data into structured relational tables.
8. To perform data analysis using SQL queries in Snowflake.
9. To visualize weather data using Python and Matplotlib.
10. To demonstrate an end-to-end cloud-based data engineering pipeline.

SYSTEM ARCHITECTURE

The proposed system consists of multiple cloud services working together to collect, process, store, and analyze weather data.

OpenWeather API



AWS Lambda (Data Ingestion)



Amazon DynamoDB



DynamoDB Streams



AWS Lambda (Validation & Classification)



Amazon S3



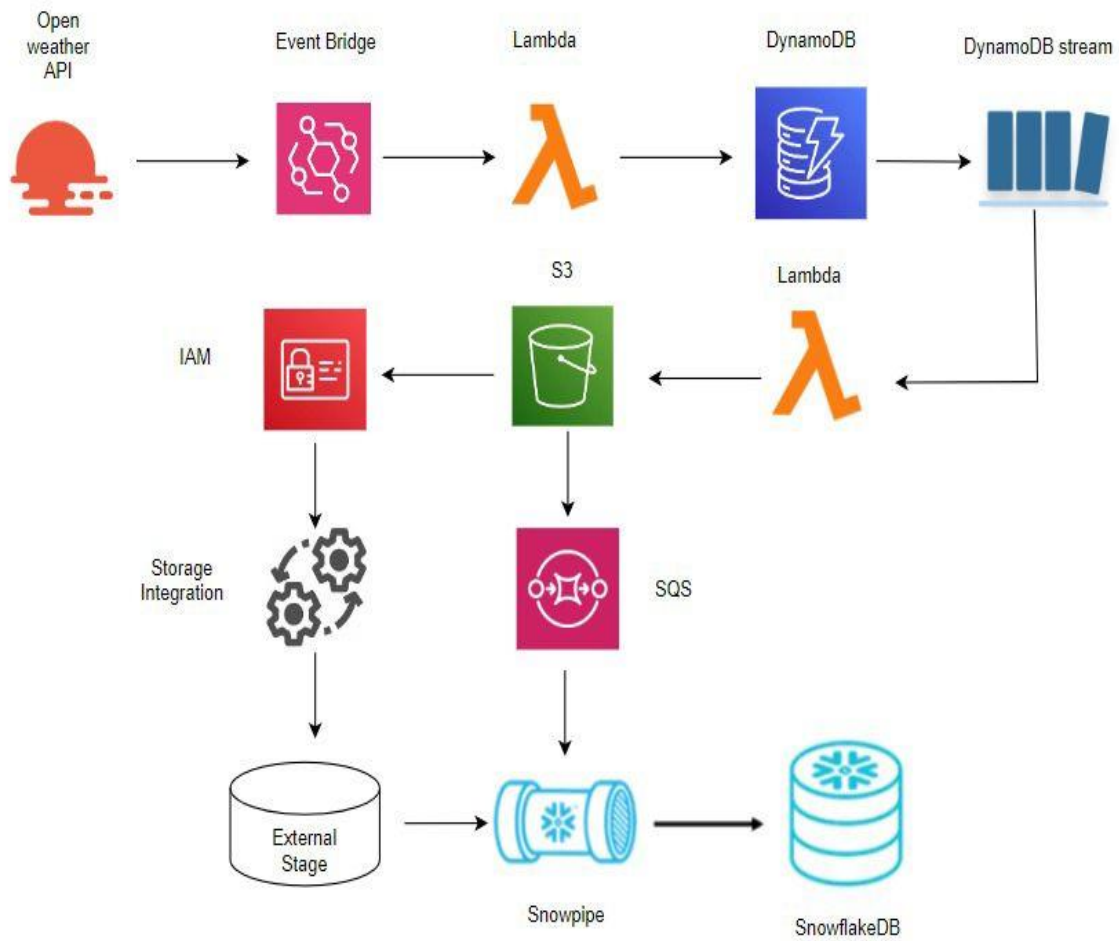
Snowflake Stage



Snowflake Tables



Python Analytics & Visualization



TECHNOLOGIES USED

1. OpenWeather API
 - Used to fetch real-time weather information.
2. AWS Lambda
 - Serverless computing service used for automated data ingestion and processing.
3. Amazon DynamoDB
 - NoSQL database used to store weather records.
4. DynamoDB Streams
 - Captures changes in DynamoDB and triggers downstream processing.
5. Amazon S3
 - Object storage service used as a data lake for storing weather data files.
6. Snowflake
 - Cloud-based data warehouse used for data loading, transformation, and analytics.
7. SQL
 - Used to query and analyze weather data in Snowflake.
8. Python
 - Used for data extraction, processing, and visualization.
9. Pandas
 - Python library used for data manipulation and analysis.

10. Matplotlib

- Python visualization library used to generate charts and graphs.

11. Visual Studio Code

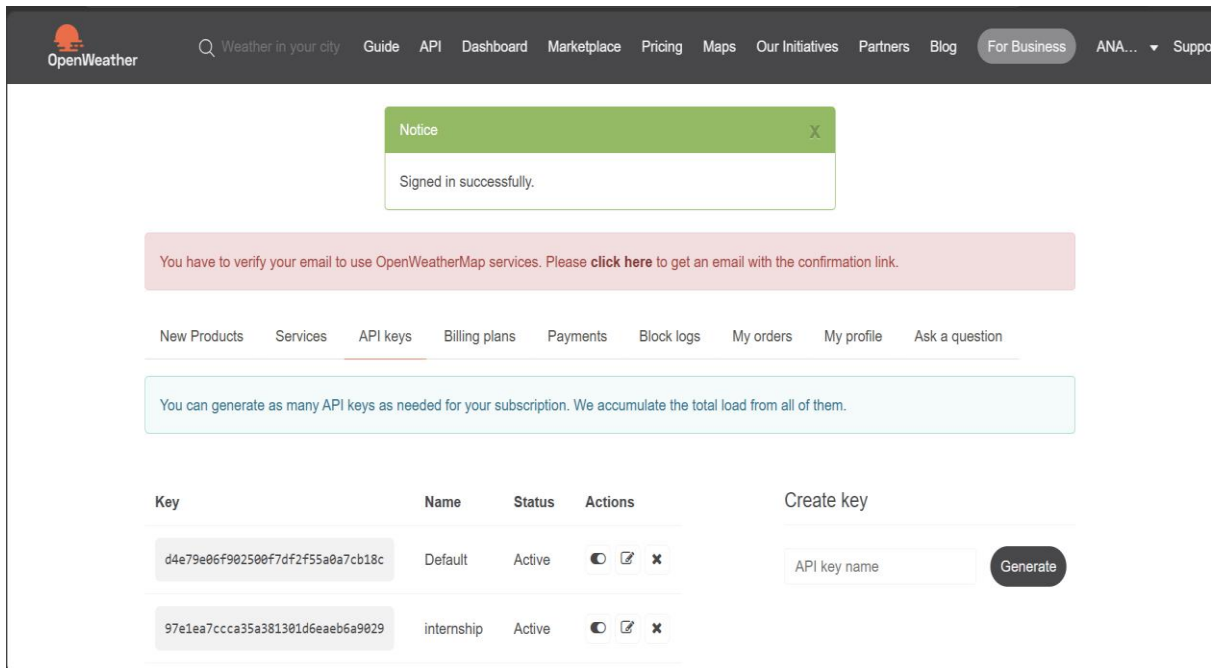
- Development environment used for coding and testing.

CHAPTER: IMPLEMENTATION AND SCREENSHOTS

1. OpenWeather API Configuration:

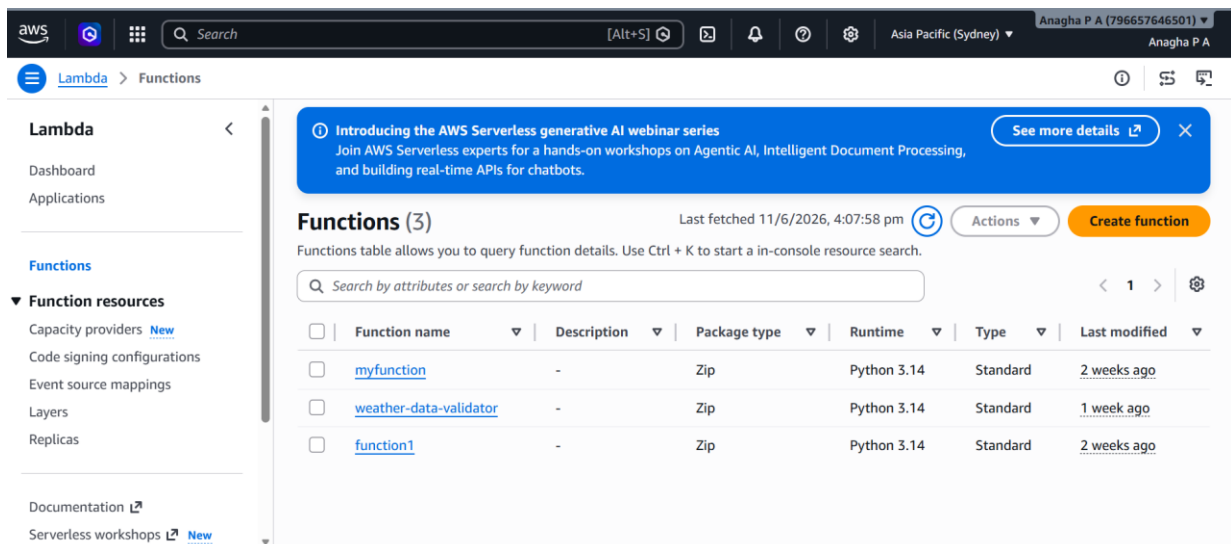
The OpenWeather API was used as the primary source of weather data.

An API key was generated and integrated into the AWS Lambda function to fetch real-time weather information for selected cities.



2. AWS Lambda Function:

A Lambda function was developed to retrieve weather information from the OpenWeather API. The function processes the API response and stores the extracted weather details in DynamoDB.

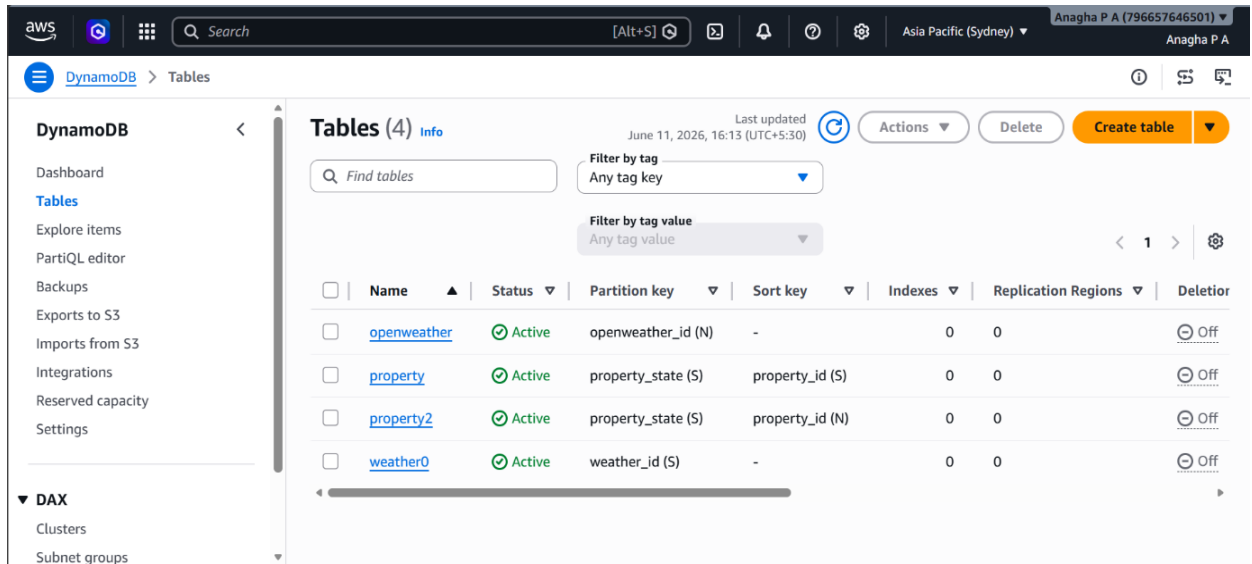


3. DynamoDB Table Creation

Table Name: weather0

Partition Key: weather_id

Amazon DynamoDB was used as the primary NoSQL database to store weather records generated by the Lambda function.

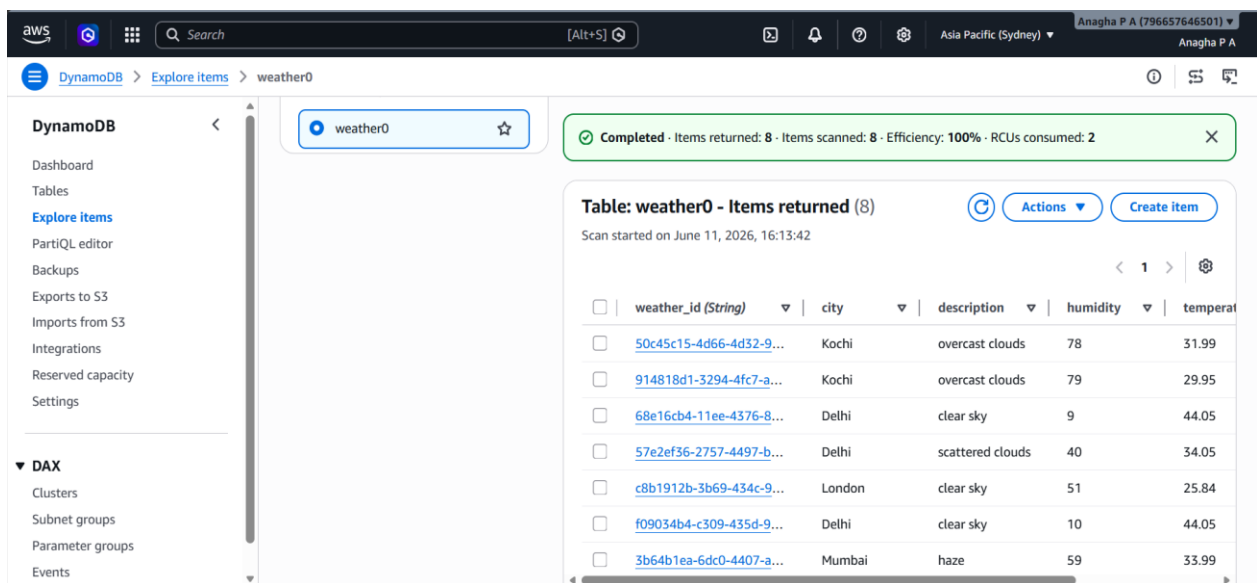


The screenshot shows the AWS DynamoDB console. On the left is a navigation menu with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, Settings, DAX, Clusters, and Subnet groups. The main area is titled 'Tables (4)' and shows a list of tables. The table 'weather0' is highlighted, showing its partition key 'weather_id' and sort key '-'. The table is active and has 0 indexes and 0 replication regions.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection
openweather	Active	openweather_id (N)	-	0	0	Off
property	Active	property_state (S)	property_id (S)	0	0	Off
property2	Active	property_state (S)	property_id (N)	0	0	Off
weather0	Active	weather_id (S)	-	0	0	Off

4. DynamoDB Records

The table stores weather information such as city name, temperature, humidity, weather description, timestamp, and unique weather identifiers.

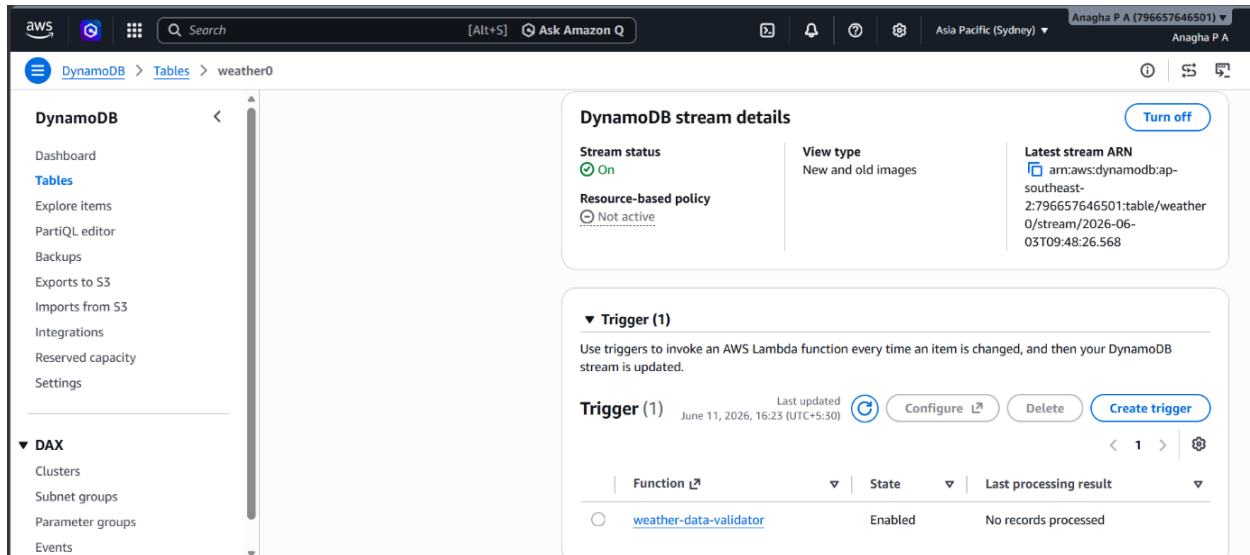


The screenshot shows the AWS DynamoDB console 'Explore items' page for the 'weather0' table. A green banner at the top indicates 'Completed - Items returned: 8 - Items scanned: 8 - Efficiency: 100% - RCUs consumed: 2'. Below this, a table lists 8 items with their unique weather IDs, cities, descriptions, humidity, and temperature.

weather_id (String)	city	description	humidity	temperature
50c45c15-4d66-4d32-9...	Kochi	overcast clouds	78	31.99
914818d1-3294-4fc7-a...	Kochi	overcast clouds	79	29.95
68e16cb4-11ee-4376-b...	Delhi	clear sky	9	44.05
57e2ef36-2757-4497-b...	Delhi	scattered clouds	40	34.05
c8b1912b-3b69-434c-9...	London	clear sky	51	25.84
f09034b4-c309-435d-9...	Delhi	clear sky	10	44.05
3b64b1ea-6dc0-4407-a...	Mumbai	haze	59	33.99

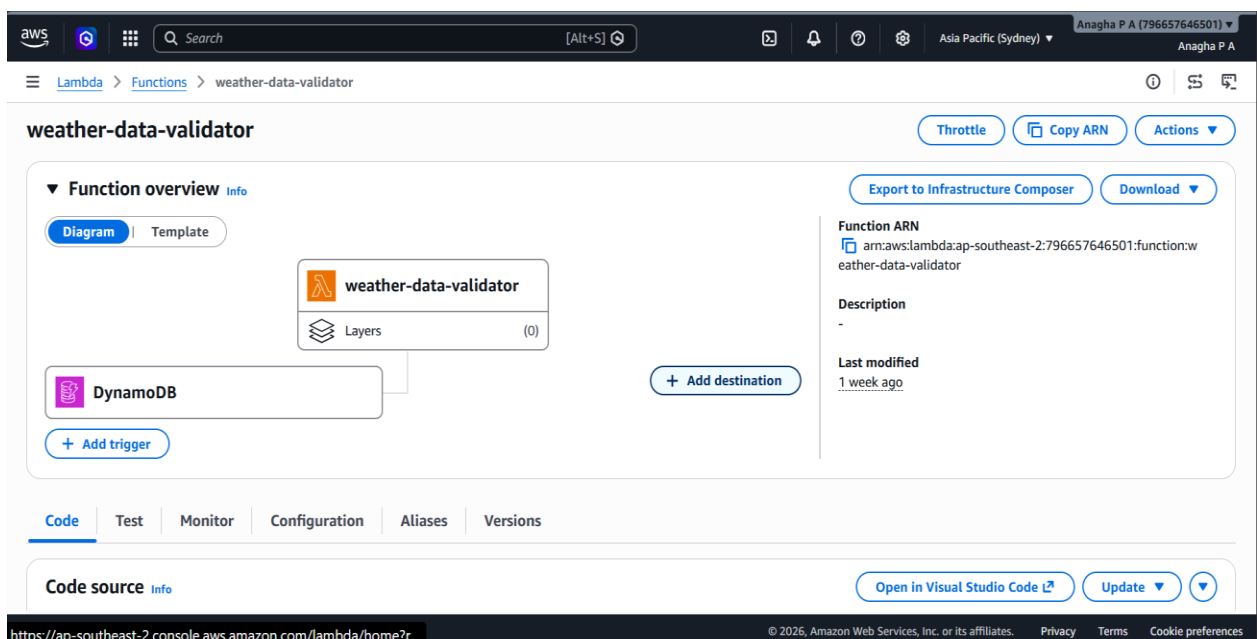
5. DynamoDB Streams Configuration

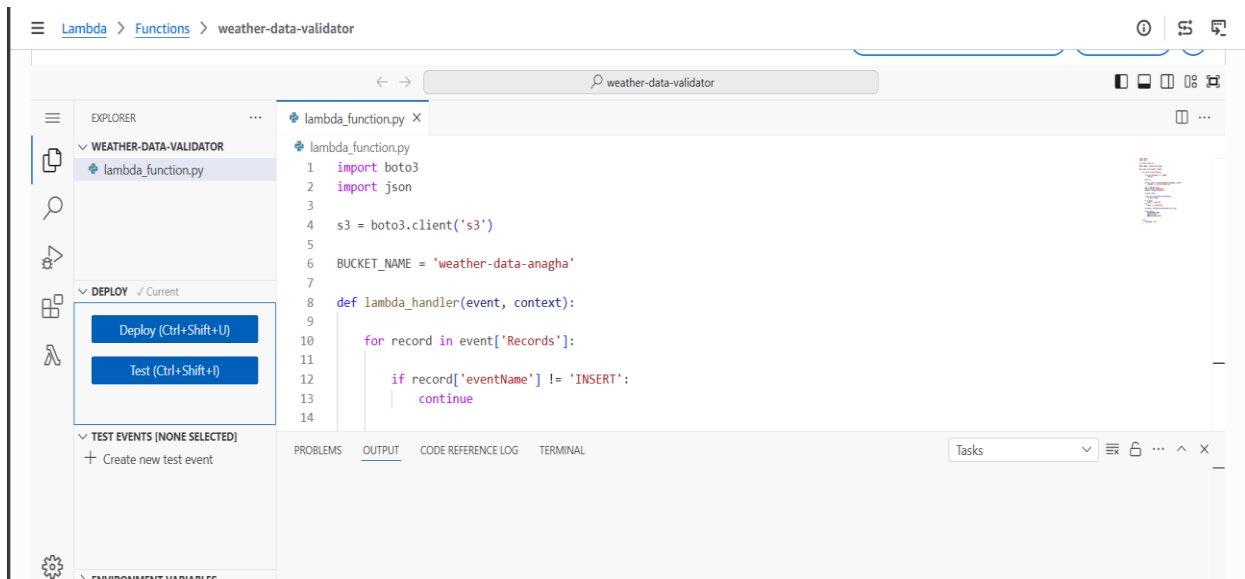
DynamoDB Streams was enabled to capture changes made to the weather table and automatically trigger downstream processing.



6. Validation Lambda Function

A second Lambda function was configured to validate incoming records. Records with missing values or duplicates were classified separately.

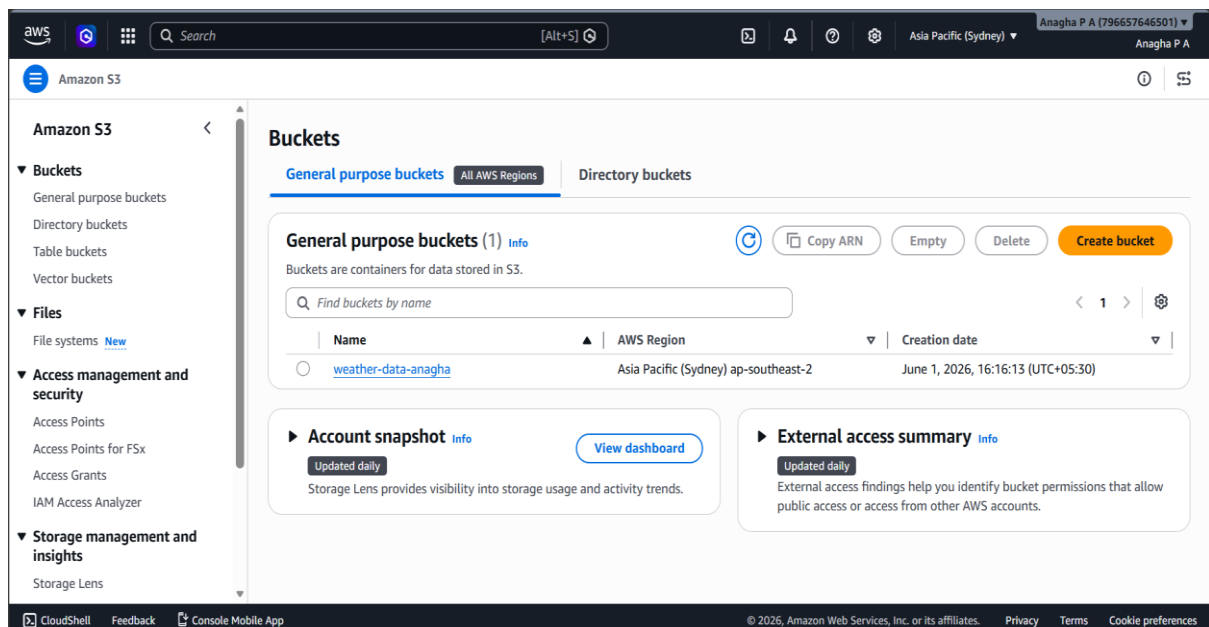




7. Amazon S3 Bucket

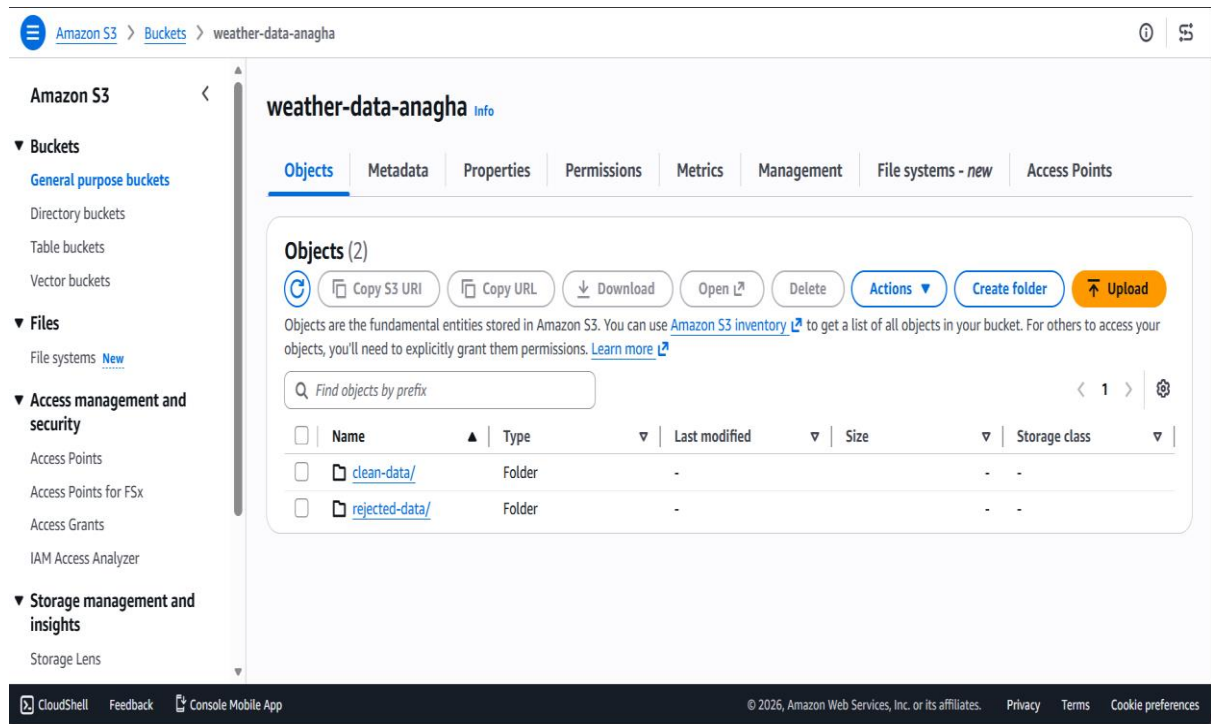
Bucket Name: weather-data-anagha

Amazon S3 serves as the centralized storage layer where validated weather records are stored in JSON format.



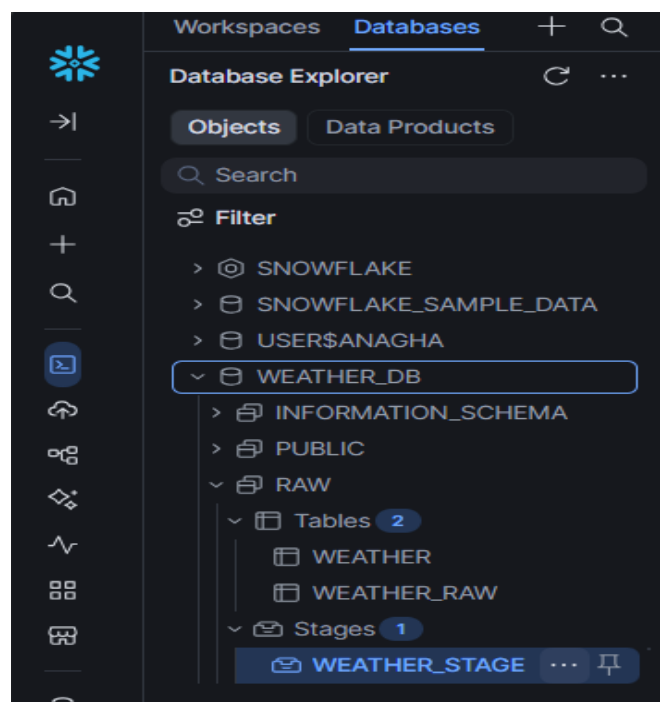
8. S3 Folder Structure

Weather records are categorized into clean and rejected datasets and stored in separate folders for easier management and analytics.



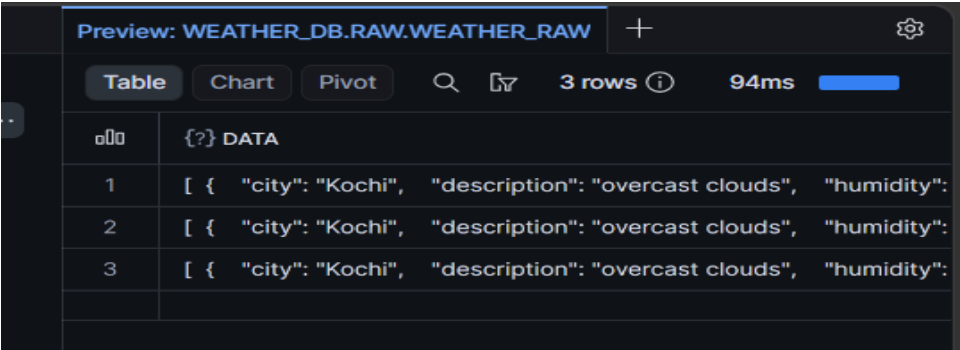
9. Snowflake Database and stage Creation

A Snowflake database was created to store and analyze weather data received from Amazon S3. A Snowflake stage was configured to load JSON files from Amazon S3 into Snowflake.



10. WEATHER_RAW Table

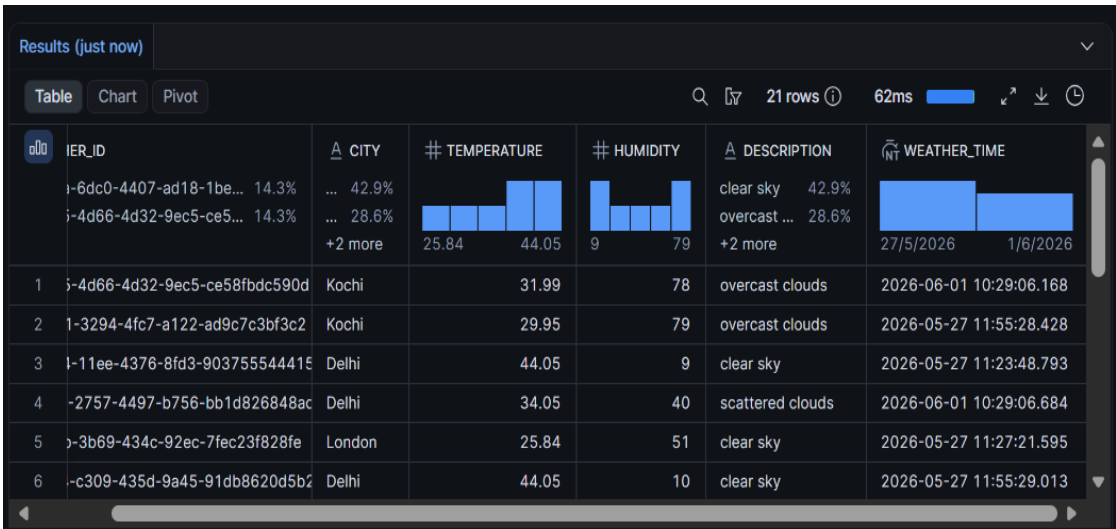
The WEATHER_RAW table stores weather data in semi-structured JSON format before transformation.



	{?} DATA
1	[{ "city": "Kochi", "description": "overcast clouds", "humidity":
2	[{ "city": "Kochi", "description": "overcast clouds", "humidity":
3	[{ "city": "Kochi", "description": "overcast clouds", "humidity":

11. WEATHER Table

Semi-structured JSON data was transformed into structured columns using Snowflake's FLATTEN function.

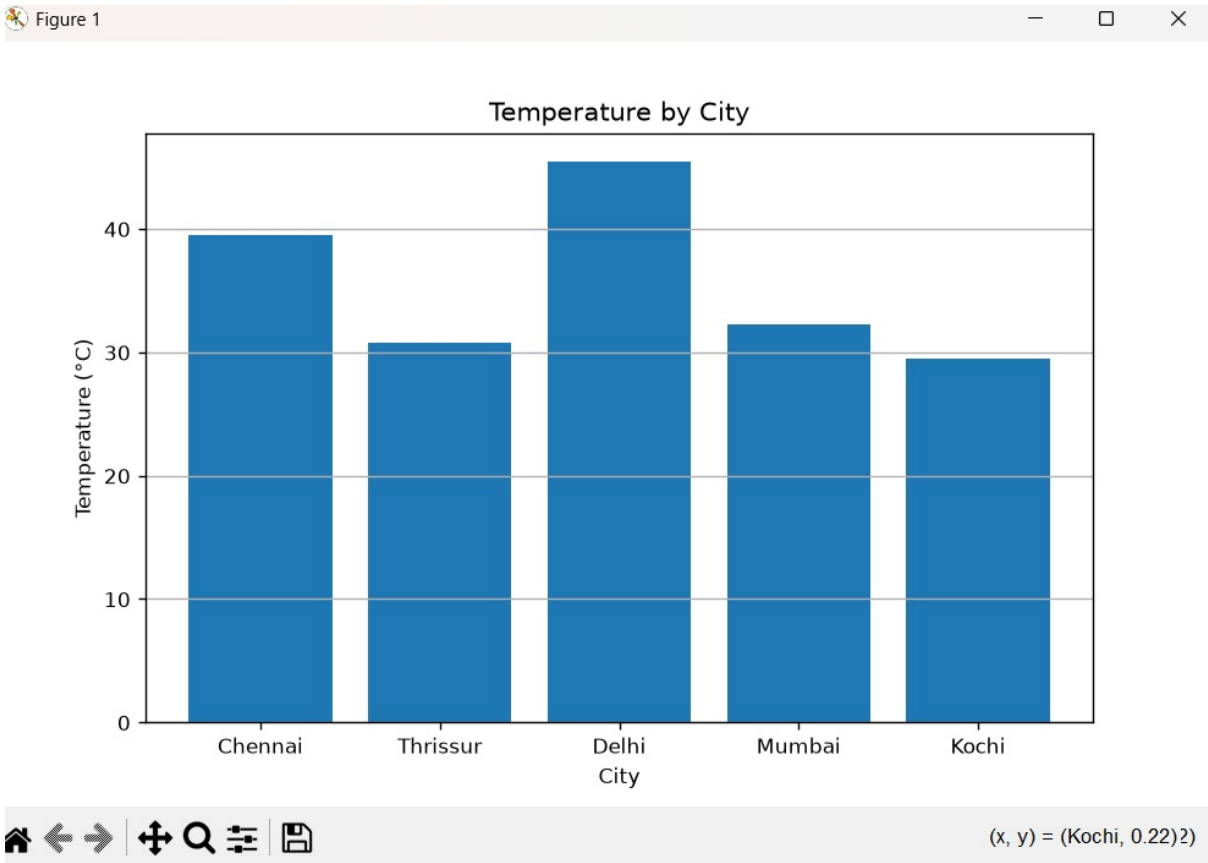


	ID	CITY	TEMPERATURE	HUMIDITY	DESCRIPTION	WEATHER_TIME
1	6dc0-4407-ad18-1be...	Kochi	31.99	78	overcast clouds	2026-06-01 10:29:06.168
2	1-3294-4fc7-a122-ad9c7c3bf3c2	Kochi	29.95	79	overcast clouds	2026-05-27 11:55:28.428
3	1-11ee-4376-8fd3-903755544415	Delhi	44.05	9	clear sky	2026-05-27 11:23:48.793
4	-2757-4497-b756-bb1d826848ac	Delhi	34.05	40	scattered clouds	2026-06-01 10:29:06.684
5	-3b69-434c-92ec-7fec23f828fe	London	25.84	51	clear sky	2026-05-27 11:27:21.595
6	-c309-435d-9a45-91db8620d5b2	Delhi	44.05	10	clear sky	2026-05-27 11:55:29.013

12.TEMPERATURE ANALYSIS CHART

Description:

The chart compares temperatures recorded across different cities.



CHAPTER: RESULTS AND DISCUSSION

The weather data pipeline was successfully implemented using AWS services, Snowflake, and Python. Real-time weather data was collected from the OpenWeather API and automatically processed through AWS Lambda functions. The collected records were stored in DynamoDB and validated using DynamoDB Streams and a secondary Lambda function.

The validated records were stored in Amazon S3 in JSON format and subsequently loaded into Snowflake for data warehousing and analytics. Semi-structured JSON data was transformed into structured relational tables using Snowflake SQL operations.

Several analytical queries were executed to extract insights such as average temperature by city, humidity levels, and weather conditions. Python libraries including Pandas and Matplotlib were used to generate graphical visualizations of the weather data.

The implementation demonstrates a complete cloud-based data engineering workflow, highlighting automation, scalability, and efficient data processing capabilities.

CHAPTER: CONCLUSION

This project successfully implemented an end-to-end weather data pipeline using AWS, Snowflake, and Python technologies. Weather information was collected from the OpenWeather API, processed using AWS Lambda, stored in DynamoDB, validated through event-driven processing, and archived in Amazon S3.

The processed data was integrated with Snowflake for warehousing and analytics purposes. SQL queries were used to derive meaningful insights, while Python and Matplotlib were utilized to visualize weather trends and comparisons.

The project demonstrates the practical application of cloud computing, data engineering, data warehousing, and analytics concepts. It provides a scalable and automated solution for collecting, processing, storing, and analyzing weather data in a modern cloud environment.

CHAPTER: REFERENCES

1. OpenWeather API Documentation
<https://openweathermap.org/api>
2. Amazon Web Services (AWS) Documentation
<https://docs.aws.amazon.com>
3. AWS Lambda Documentation
<https://docs.aws.amazon.com/lambda>
4. Amazon DynamoDB Documentation
<https://docs.aws.amazon.com/dynamodb>
5. DynamoDB Streams Documentation
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>
6. Amazon S3 Documentation
<https://docs.aws.amazon.com/s3>
7. Snowflake Documentation
<https://docs.snowflake.com>
8. Snowflake Data Loading Documentation
<https://docs.snowflake.com/en/user-guide/data-load-overview>
9. Snowflake SQL Reference Guide
<https://docs.snowflake.com/en/sql-reference>
10. Python Official Documentation
<https://docs.python.org/3>
11. Pandas Documentation
<https://pandas.pydata.org/docs>
12. Matplotlib Documentation
<https://matplotlib.org/stable/index.html>
13. Visual Studio Code Documentation
<https://code.visualstudio.com/docs>
14. JSON Documentation
<https://www.json.org>
15. REST API Design Guide
<https://restfulapi.net>
16. Snowflake Python Connector Documentation
<https://docs.snowflake.com/en/developer-guide/python-connector>
17. Data Engineering Fundamentals
Kleppmann, Martin. Designing Data-Intensive Applications. O'Reilly Media, 2017.